Course: Logic Programming
Programming Language: Prolog
Assessment Type: Project
Project Title: Design and Implementation of a Knowledge-Based Problem-Solving System Using Logic Programming

1. Project Description

Students are required to design, develop, test, and document a knowledge-based application using logic programming techniques. The application should represent knowledge using facts and rules and use logical inference to answer queries or solve problems within a selected domain.

Students will apply the concepts covered in class, including facts, predicates, rules, queries, unification, backtracking, recursion, lists, arithmetic, and other relevant Prolog features.

Students may select a suitable problem from education, agriculture, healthcare information, business, transportation, environmental management, troubleshooting, or another domain where knowledge can be represented using logical relationships and rules.

2. Project Objectives

By the end of the project, students should be able to:

1. Explain the principles of logic programming.
2. Represent knowledge using facts and predicates.
3. Define logical rules to derive new information.
4. Write queries to retrieve information from a knowledge base.
5. Apply unification and backtracking to solve problems.
6. Implement recursive predicates and list-processing operations.
7. Apply arithmetic and comparison operators in Prolog.
8. Use appropriate techniques for organizing and querying knowledge.
9. Develop a working knowledge-based application.
10. Test, document, and explain the logic used by the application.

3. Project Requirements

The application must demonstrate appropriate use of the following concepts:

1. **Facts and Predicates:** Represent information about entities, properties, and relationships.
2. **Rules:** Define logical relationships and conditions for deriving conclusions. 3. **Queries:** Allow users to retrieve information or test whether relationships hold. 4. **Unification:** Demonstrate how Prolog matches terms and binds variables. 5. **Backtracking:** Use alternative solutions to find matching results.
6. **Recursion:** Implement at least one meaningful recursive predicate where appropriate.
7. **Lists:** Store and process collections of related information.
8. **Arithmetic and Comparisons:** Apply arithmetic operations and comparison predicates where required.
9. **Knowledge-Based Reasoning:** Use facts and rules to derive answers or

recommendations.

10. **Input and Output:** Provide an appropriate way for users to interact with the application.
11. **Testing:** Test valid queries, invalid inputs, edge cases, and expected results. 12. **Documentation:** Explain the knowledge representation, rules, queries, and reasoning process.

4. Project Stages

Stage 1: Problem Identification and Proposal

- Identify a problem suitable for logic programming.
- Define the problem statement, objectives, and scope.
- Identify the knowledge required to solve the problem.
- Describe the intended users and expected outputs.
- Explain why logic programming is suitable for the problem.

Deliverable: Project proposal and problem specification.

Stage 2: Knowledge Representation Using Facts

- Identify the entities and relationships within the selected domain.
- Define appropriate predicates.
- Create a knowledge base containing relevant facts.
- Write queries to retrieve information from the knowledge base.

Deliverable: Prolog source file containing facts and working queries.

Stage 3: Rules and Logical Inference

- Define rules based on relationships and conditions.
- Combine facts and rules to derive conclusions.
- Write queries that demonstrate logical inference.
- Test the rules with different combinations of facts.

Deliverable: Knowledge base containing facts, rules, and demonstrated inference queries.

Stage 4: Unification and Backtracking

- Demonstrate variable binding through unification.
- Write queries that return multiple solutions.
- Explain how Prolog searches for alternative solutions.
- Apply backtracking to retrieve relevant answers.

Deliverable: Working queries and a report explaining unification and backtracking.

Stage 5: Recursion and Lists

- Implement recursive predicates appropriate to the problem.

- Use lists to represent collections of information.
- Implement suitable list-processing operations.
- Test recursive predicates using different inputs.

Deliverable: Recursive predicates and list-processing functionality.

Stage 6: Arithmetic and Advanced Rules

- Apply arithmetic and comparison predicates where necessary.
- Implement more complex rules involving multiple conditions.
- Apply negation as failure or the cut operator where appropriate.
- Evaluate how additional rules improve the application's reasoning capabilities.

Deliverable: Extended knowledge base with advanced rules and tested queries.

Stage 7: Application Integration and Testing

- Integrate the facts, rules, and predicates into a coherent application. • Organize the source code into logical sections or modules where appropriate. • Provide an appropriate user interaction mechanism.
- Test the application against defined requirements.
- Correct logical errors and unexpected behaviour.

Deliverable: Integrated and tested Prolog application.

Stage 8: Final Report and Presentation

- Prepare the complete project report.
- Explain the knowledge representation and inference process.
- Demonstrate the application's main features.
- Present test cases and results.
- Discuss limitations and possible improvements.
- Defend the design decisions during an oral presentation.

Deliverable: Complete Prolog source code, final report, and

presentation. **5. Expected Project Deliverables**

Each student or group must submit:

1. Project proposal and problem specification.
2. Knowledge representation design.
3. Prolog facts and rules.
4. Queries demonstrating the application's functionality.
5. Implementation of unification, backtracking, recursion, and lists where appropriate.
6. Source code for the completed application.
7. Test cases and test results.
8. Technical documentation and user instructions.
9. Final project report.

10. Project demonstration and oral defense.

6. Submission Guidelines

- The project may be completed individually or in groups.
- Prolog must be the primary programming language.
- The knowledge base must contain meaningful facts and rules relevant to the selected problem.
- Students must demonstrate the application through working queries and explain the results.
- Source code must be clearly organized and appropriately commented. •

All major predicates must be tested with suitable examples.

• Students must be able to explain how the system derives its conclusions. • External sources and knowledge used in the project must be acknowledged. • A video link on the google drive should be submitted entailing your explanation of the project.